Disciplina: 11927 – Introdução à Compilação
Profa. Valéria

2º Trabalho Prático

Geração de código MEPA para a linguagem Tascal

Objetivo

O 2º trabalho prático consiste na complementação da implementação do compilador da linguagem Tascal (*Tiny Pascal*). Após às análises, o compilador deverá gerar código intermediário para ser interpretado por uma máquina virtual denominada MEPA (Kowaltowski, 1983). Uma implementação dessa máquina será disponibilizada junto com os arquivos de apoio a este trabalho.

O trabalho poderá ser desenvolvido nas seguintes linguagens: C, C++ ou Python. Será permitido o uso de ferramentas de geração automática de *scanners* e *parsers*, como Flex[1], Bison[2] (para linguagem C/C++) e PLY[3] (para linguagem Python).

O compilador deverá ser executado a partir da linha de comando, passando como argumento o arquivo de código-fonte em Tascal a ser analisado.

Normas para desenvolvimento e entrega do trabalho

O trabalho poderá ser desenvolvido **por uma equipe de duas pessoas**.

A submissão do trabalho deverá ser feita exclusivamente pelo *Classroom*. **O prazo final para a entrega será até às 19 horas do dia 10/12/2025**.

Deverão ser entregues os seguintes artefatos:

- **Código-fonte** do compilador (compactado no caso de vários arquivos separados);
- **Relatório técnico** curto (máx. 3 páginas), explicando:
  - Organização básica do código;
  - Etapas implementadas;
  - Descrição e justificativas de possíveis etapas não cumpridas.

Avaliação do trabalho

O código-fonte do compilador será avaliado de acordo com sua organização, correção e requisitos atendidos. Para isso, um conjunto de programas escritos em Tascal será utilizado para verificar as etapas que foram cumpridas no desenvolvimento do compilador. Seu compilador deve aceitar corretamente programas válidos e rejeitar programas inválidos. As mensagens de erro devem indicar a linha e a natureza do erro (se léxico, sintático ou semântico). **Neste segundo trabalho, não será avaliada a qualidade das mensagens de erro, visto que o foco principal será a geração de código MEPA.**

[1] https://westes.github.io/flex/manual/
[2] https://www.gnu.org/software/bison/manual/
[3] https://ply.readthedocs.io/en/latest/index.html

Também fará parte da avaliação do trabalho, uma apresentação sobre sua implementação, a ser realizada pela equipe nos dias 10/12/2025 e 15/12/2025, conforme agendamento prévio.

Geração de código

O funcionamento da máquina MEPA e suas instruções serão explicadas em aula, mas também podem ser consultadas em Kowaltowski (1983). Não é um requisito do trabalho que o *parser* gere uma Árvore Sintática Abstrata (AST), mas é uma recomendação. O objetivo de se construir uma AST explícita em vez de utilizar as ações semânticas do *parser* para a geração de código é separar as diferentes etapas de compilação. Dessa forma, o código ficará mais organizado (modularizado), facilitando sua manutenção e depuração.

Casos omissos

Quaisquer casos relacionados ao desenvolvimento deste trabalho que não foram esclarecidos nesta especificação deverão ser arguidos diretamente com a professora.

Referências

KOWALTOWSKI, T. **Implementação de Linguagens de Programação**. Rio de Janeiro: Guanabara Dois, 1983.

Anexo I - Gramática da linguagem Tascal

Regras léxicas para identificadores e números

```
<número> ::= <dígito> { <dígito> }

<dígito> ::= 0-9

<identificador> ::= <letra> { <letra> | <dígito> | '_'}

<letra> ::= a-zA-Z
```

Regras de produção da gramática

```
<programa> ::=
    'program' <identificador> ';' <bloco> '.'

<bloco> ::=
    [ <seção_declaração_variáveis> ]
    <comando_composto>

<seção_declaração_variáveis> ::=
    'var' <declaração_variáveis> ';' { <declaração_variáveis> ';' }

<declaração_variáveis> ::=
    <lista_identificadores> ':' <tipo>

<lista_identificadores> ::=
    <identificador> { ',' <identificador> }

<tipo> ::=
    'boolean' | 'integer'

<comando_composto> ::=
    'begin' <comando> { ';' <comando> } 'end'

<comando> ::=
      <atribuição>
    | <condicional>
    | <repetição>
    | <leitura>
    | <escrita>
    | <comando_composto>

<atribuição> ::=
    <identificador> ':=' <expressão>
```

```
<condicional> ::=
    'if' <expressão> 'then' <comando> [ 'else' <comando> ]

<repetição> ::=
    'while' <expressão> 'do' <comando>

<leitura> ::=
    'read' '(' <lista_identificadores> ')'

<escrita> ::=
    'write' '(' <lista_expressões> ')'

<lista_expressões> ::=
    <expressão> { ',' <expressão> }

<expressão> ::=
    <expressão_simples> [ <relação> <expressão_simples> ]

<relação> ::=
    '=' | '<>' | '<' | '<=' | '>' | '>='

<expressão_simples> ::=
    <termo> { ( '+' | '-' | 'or' ) <termo> }

<termo> ::=
    <fator> { ( '*' | 'div' | 'and' ) <fator> }

<fator> ::=
      <variável>
    | <número>
    | <lógico>
    | '(' <expressão> ')'
    | 'not' <fator>
    | '-' <fator>

<variável> ::=
    <identificador>

<lógico> ::=
      'false'
    | 'true'
```